Toward A Hierarchical Tower-Rover Architecture Under Compute and Bandwidth Constraints for Multi-Agent Deep Space Exploration

Jayden Vasquez
Ohio State University, Columbus, OH, USA
vasquez.208@osu.edu

***Abstract*—Robotic exploration of the deep ocean and deep space shares common constraints: missions are frequently single-shot, communication is limited or delayed, and on-board compute must be conserved rather than assumed abundant. We describe the architecture and current implementation of a hierarchical multi-robot exploration platform built around these constraints. We propose a compute-capable observation tower that fuses multi-robot sensing into a shared map and issues high-level task goals, and a set of resource-constrained ground rovers that execute locally and fall back to autonomous behavior when the tower link is degraded or lost. We report on the working implementation of the platform's rover-side perception, control, and communication layers across three rovers. This includes a bandwidth-bounded camera pipeline and a scheme for sharing the rover's single camera, which only one process can open at a time, between the detection loop and remote monitoring, and outline the model-compression work required to move the tower's large vision-language-action (VLA) planner onto the rovers themselves. We report implementation status and design rationale rather than field results, and use it to motivate the model-compression roadmap described in Section VI.**

***Index Terms*—multi-robot systems, one-shot exploration, edge autonomy, bandwidth-constrained communication, hierarchical architecture, model compression**

I. Introduction

Deep-ocean and deep-space robotic exploration have historically progressed through iterative missions, with human operators in the loop for planning and recovery [1], [2]. Flagship-class missions such as the Mars Science Laboratory and Mars 2020 rovers have demonstrated substantial onboard autonomy, including autonomous navigation and onboard science-target selection under communication delay [2], but this capability has been developed and validated on platforms with compute, power, and mass budgets far larger than those available to small, resource-constrained exploration platforms. That model breaks down when a mission must succeed in a single deployment, since scarce research-vessel time or launch windows very rarely allow for a second attempt. Even within that single deployment, the communication link back to a human is often slow, weak, or delayed by minutes to hours, so the system cannot rely on the kind of real-time human correction earlier missions used. Sparse sensing compounds this, forcing robots to act under partial observability, so these systems must act on local information when the link to a higher-level planner is thin or absent.

We are building a small-scale model of this problem: one observation tower with access to a large planning model, and several resource-constrained rovers that must operate independently when tower communication degrades. The tower concentrates scarce compute in one elevated, better-powered platform with a wider view of the workspace, mirroring the orbiter/lander and relay/rover splits used in planetary exploration. The rover compute budget (a Raspberry Pi 5 with an NPU accelerator) is representative of small exploration platforms.

We target a vision-language planner because mission goals are naturally open-ended instructions (e.g., "locate and approach unusual terrain features") rather than a fixed target list, and a language-conditioned planner can take new goals without retraining [3].

Tower-to-rover communication is not assumed reliable. Links between a fixed relay and moving robots degrade with distance, terrain, and interference, just as landers lose contact with rovers through terrain shadowing or scheduled downlink windows. A rover that stalls whenever the link drops would lose most of its operating time, so we treat link loss as an expected state rather than a rare failure.

Compression matters because the tower's planner is too large for rover hardware, and a single-deployment mission cannot retrain or swap models mid-mission. Multiple low-cost rovers matter because they cover more ground and tolerate losing a unit, which is only affordable if each rover is cheap.

Our contributions are (1) the hierarchical system architecture, (2) a working implementation of its rover-side perception, control, and communication layers on three rovers, and (3) the compression roadmap that connects this implementation to the platform's goal of pushing tower-level intelligence onto bandwidth- and compute-constrained rovers.

II. Related Work

Hierarchical relay architectures are well established in planetary exploration, where an orbiter or lander with better communication and compute relays goals and shared maps to surface assets with tighter power and mass budgets. The Cooperative Autonomous Distributed Robotic Exploration (CADRE) multi-rover demonstration follows this pattern, and we intend to draw on its launch structure, SLAM message layout, and rover namespacing conventions [4]. Communication-restricted

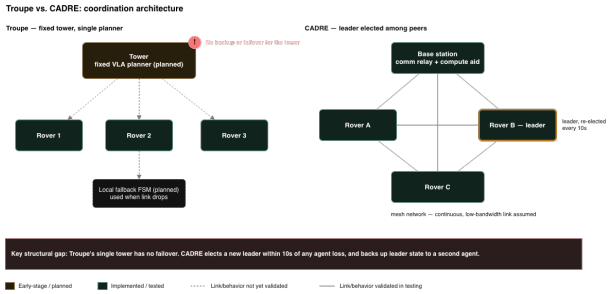


Fig. 1. Comparison between the proposed tower-rover architecture and CADRE's leader-election architecture

multi-robot exploration is a long-studied problem [5] and remains active, with recent surveys covering the range of coordination strategies used under tight bandwidth [6]. Closest to our tower-rover setting is recent work on learned routing policies for rovers relaying data to a lander under intermittent connectivity in lunar delay-tolerant networks [7], built around the same CADRE-style mission concept our architecture is modeled on, though at the level of network routing rather than system architecture and on-rover perception. On the mapping side, Isaac ROS Nvblox provides GPU-accelerated ESDF fusion from streamed depth and pose [8], and Nav2 consumes such costmaps for planning [9]. Both are planned for use at the tower level in our architecture but are not yet integrated. Frontier-based exploration [10] and decentralized multi-robot SLAM [11] inform the rover-side search behavior we plan to add.

On the perception side, a recent survey documents the range of techniques used to deploy vision-language models on resource-constrained edge hardware, including pruning, quantization, knowledge distillation, and specialized hardware support [12]. Separate work targets running vision-language-action (VLA) models specifically on CPU and edge-bound robots [13]. Separate literature addresses how to compress such models for constrained inference, such as joint multi-precision training [14], nested elastic subnetworks [15], hardware-aware mixed-precision search [16], and VLA-specific dynamic quantization [17], [18]. These techniques are the intended target of the compression work described in Section VI.

III. System Architecture

The platform splits responsibility between one tower and multiple rovers, summarized in Fig. 2.

The tower's planner is intended to be a large VLA model whose output is converted into a structured mission plan and dispatched to rovers as discrete task goals (e.g., approach a coordinate, orient toward a detected object). Rovers maintain their own local state estimate and task queue and are designed to continue executing that queue, falling back to a default holding behavior, if the tower link drops. This fallback behavior and the tower-side VLA planner are architectural design targets. The implementation reported below covers the rover-

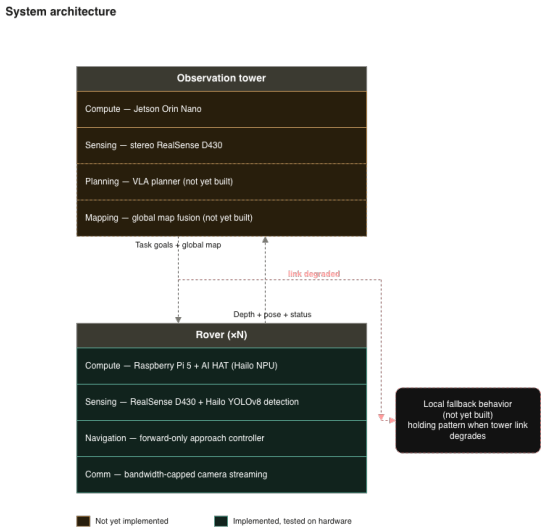


Fig. 2. Tower vs. rover roles and hardware.

side perception and control loop and the communication layer that will eventually carry tower-issued goals.

We refer to the platform as Troupe. As Fig. 1 highlights, the tower is a single point of failure. CADRE re-elects a leader among its rovers within seconds of losing one and backs up leader state to a second agent [4], whereas our design has no tower failover. We accept this deliberately. Rovers are designed to treat tower loss the same way as a degraded link: they continue their local task queue and then hold. Losing the tower therefore reduces the team to independent local operation rather than halting it. Adding failover would require either a second tower or rover hardware capable of running the planner, which the compression roadmap in Section VI is intended to enable.

IV. Methodology

The components below run end-to-end on the target hardware (rover compute unit, low-level actuation controller, and camera). Three identical rovers have been built and each tested individually; they have not yet been operated together under tower coordination, and the link-loss fallback is not yet implemented.

A. Perception and Approach Control

A rover-side controller fuses NPU-accelerated object detection with aligned stereo depth to approach and hold station near a detected target. Detections are filtered by confidence and matched to a depth estimate taken as the median valid depth in a small window around the detection's center, which is more robust to single-pixel depth dropout than a point sample. The resulting bearing and range error drive a forward-only approach controller, where the rover advances while the target is farther than a target standoff distance plus a dead-band, holds station inside the dead-band, and never commands reverse motion, with independent decay-to-center behavior on the throttle and steering channels when no target is visible.

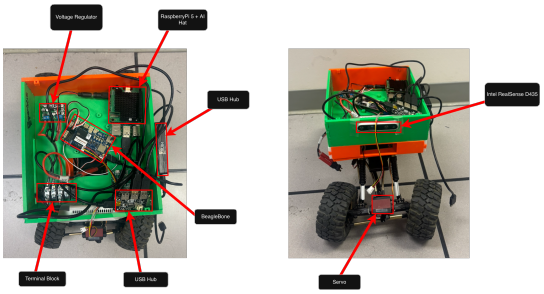


Fig. 3. Labeled rover electronics enclosure.

Commands are sent as UDP packets to a low-level servo daemon at a fixed rate, separate from how fast the vision loop runs, so a slow camera frame does not stall the actuation update. The detector currently used in this loop is YOLOv8 from the Hailo Model Zoo, an off-the-shelf, precompiled model running on the rover's NPU accelerator. It is not a model we compressed ourselves, and its replacement with a distilled tower-derived model is future work (Section VI).

B. Manual Drive Control

A keyboard teleoperation controller supports two modes. With a USB keyboard on the rover, held keys give continuous throttle and steering; over SSH, where held keys are unreliable, it uses tap-to-toggle steering and a timeout-based throttle hold. Both modes send the same UDP commands as the approach controller, so manual and autonomous control are interchangeable downstream.

C. Sharing One Camera Between Detection and Remote Monitoring

Only one process can open the RealSense camera at a time, but both the detection loop and remote monitoring want frames from it, and the ROS side used for monitoring runs in a container that should not need direct camera access. There are two ways this is set up, and only one runs at a time: (1) a container owns the camera directly and republishes JPEG frames onto the ROS graph at a fixed rate, width, and quality or (2) the host process that already has the camera open for detection sends raw JPEG frames over a local TCP connection to a separate container that has no camera access and just republishes them onto the ROS graph. Before starting the detection loop, the code checks whether a conflicting camera process is already running in Docker and exits with a clear error if so, rather than failing on a generic device-busy error. Both streaming paths cap resolution, rate, and JPEG quality before frames leave the process that captured them, which keeps the stream small for a constrained link.

V. Discussion and Conclusion

Each of the three rovers runs this loop end-to-end today: it detects a target, estimates range, approaches and holds station, hands off to manual control, and streams compressed video. We have not yet measured bandwidth, latency, or approach success; these are the first quantities we will report. The tower-side planner, map fusion, task dispatch, and link-loss fallback remain unimplemented, and the main gap between the implemented and architected system is the model-compression problem outlined in Section VI.

VI. Future Work

The architecture in Fig. 2 assumes the tower's large VLA planner is too large to run on rover-class hardware, and that rovers instead run a smaller, distilled model. Our plan is to (1) train rover-side models against the tower model's own outputs, following standard knowledge-distillation practice [19], (2) use joint multi-precision or nested-subnetwork training so one set of weights can serve multiple rover memory budgets [14], [15], and (3) test whether the resulting models hold up at the reduced precision Pi-class hardware requires, using hardware-aware precision selection [16] rather than one fixed precision as a baseline.

References

[1] D. R. Yoerger, M. Jakuba, A. M. Bradley, and B. Bingham, "Techniques for Deep Sea Near Bottom Survey Using an Autonomous Underwater Vehicle," *International Journal of Robotics Research*, vol. 26, no. 1, pp. 41–54, 2007.

[2] V. Verma *et al.*, "Autonomous Robotics Is Driving Perseverance Rover's Progress on Mars," *Science Robotics*, vol. 8, no. 80, 2023.

[3] M. J. Kim, K. Pertsch, S. Karamcheti, *et al.*, "OpenVLA: An Open-Source Vision-Language-Action Model," in *Proc. Conf. on Robot Learning (CoRL)*, 2024.

[4] "NASA Space ROS Simulation Challenge – CADRE Demo," *GitHub*.

[5] F. Amigoni, J. Banfi, and N. Basilico, "Multirobot Exploration of Communication-Restricted Environments: A Survey," *IEEE Intelligent Systems*, vol. 32, no. 6, pp. 48–57, 2017.

[6] C. Wang *et al.*, "Multi-Robot System for Cooperative Exploration in Unknown Environments: A Survey," *arXiv:2503.07278*, 2025.

[7] F. Lozano-Cuadra, B. Soret, M. Sanchez Net, A. Cauligi, and F. Rossi, "Learning Decentralized Routing Policies via Graph Attention-Based Multi-Agent Reinforcement Learning in Lunar Delay-Tolerant Networks," *arXiv:2510.20436*, 2025.

[8] A. Millane *et al.*, "nvblox: GPU-Accelerated Incremental Signed Distance Field Mapping," in *Proc. IEEE ICRA*, 2024.

[9] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A Navigation System," in *Proc. IEEE/RSJ IROS*, 2020.

[10] A. Topiwala, P. Inani, and A. Kathpal, "Frontier Based Exploration for Autonomous Robot," *arXiv:1806.03581*, 2018.

[11] P.-Y. Lajoie and G. Beltrame, "Swarm-SLAM: Sparse Decentralized Collaborative Simultaneous Localization and Mapping Framework for Multi-Robot Systems," *IEEE Robotics and Automation Letters*, vol. 9, no. 1, 2024.

[12] A. Sharshar, L. U. Khan, W. Ullah, and M. Guizani, "Vision-Language Models for Edge Networks: A Comprehensive Survey," *arXiv:2502.07855*, 2025.

[13] J. Williams, K. D. Gupta, R. George, and M. Sarkar, "Lite VLA: Efficient Vision-Language-Action Control on CPU-Bound Edge Robots," *arXiv:2511.05642*, 2025.

[14] Q. Jin, L. Yang, and Z. Liao, "AdaBits: Neural Network Quantization with Adaptive Bit-Widths," in *Proc. IEEE/CVF CVPR*, 2020.

[15] Devvrit, S. Kudugunta, *et al.*, "MatFormer: Nested Transformer for Elastic Inference," in *Proc. NeurIPS*, 2024.

[16] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-Aware Automated Quantization with Mixed Precision," in *Proc. IEEE/CVF CVPR*, 2019.

[17] Z. Zheng *et al.*, "DyQ-VLA: Temporal-Dynamic-Aware Quantization for Embodied Vision-Language-Action Models," *arXiv:2603.07904*, 2026.

[18] H. Fang, Y. Liu, Y. Du, L. Du, and H. Yang, "SQAP-VLA: A Synergistic Quantization-Aware Pruning Framework for High-Performance Vision-Language-Action Models," *arXiv:2509.09090*, 2025.

[19] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *arXiv:1503.02531*, 2015.